Atelier 1 SQL Complet – Requêtes, Calculs, Optimisation et Sécurité

Thème : Gestion commerciale (clients, commandes, produits, paiements)

Durée totale : 4 heures

🎓 Objectifs

✅ Concevoir une base de données relationnelle cohérente.

✅ Manipuler les **jointures**, **fonctions SQL**, **opérateurs**, et **agrégations**.

✅ Utiliser **IF()**, **LIMIT**, **DATEDIFF()**, et les **fonctions de date**.

✅ Améliorer la performance avec **index** et **vues**.

✅ Gérer les **droits d'accès (DCL)**.

---

1. Conception de la base (40 min)

1.1 Création de la base

```
CREATE DATABASE atelier_commerce;
USE atelier_commerce;
```

1.2 Création des tables

```
CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50),
  ville VARCHAR(50)
);

CREATE TABLE produits (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom_produit VARCHAR(50),
  prix_unitaire DECIMAL(10,2)
);
```

```sql
CREATE TABLE commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  date_commande DATE,
  FOREIGN KEY (client_id) REFERENCES clients(id)
);

CREATE TABLE details_commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  commande_id INT,
  produit_id INT,
  quantite INT,
  FOREIGN KEY (commande_id) REFERENCES commandes(id),
  FOREIGN KEY (produit_id) REFERENCES produits(id)
);

CREATE TABLE paiements (
  id INT AUTO_INCREMENT PRIMARY KEY,
  commande_id INT,
  montant DECIMAL(10,2),
  mode_paiement VARCHAR(30),
  date_paiement DATE,
  FOREIGN KEY (commande_id) REFERENCES commandes(id)
);
```

1.3 Insertion de données

```sql
INSERT INTO clients (nom, ville)
VALUES ('Alice', 'Paris'), ('Bob', 'Lyon'), ('Charlie', 'Toulouse'), ('Dina',
'Nice');

INSERT INTO produits (nom_produit, prix_unitaire)
VALUES ('Ordinateur', 950), ('Souris', 20), ('Clavier', 45), ('Écran', 250);

INSERT INTO commandes (client_id, date_commande)
VALUES (1, '2025-01-10'), (1, '2025-01-20'), (2, '2025-02-01'), (3, '2025-02-
10');

INSERT INTO details_commandes (commande_id, produit_id, quantite)
VALUES (1,1,1), (1,2,2), (2,3,1), (3,4,2), (4,2,3);

INSERT INTO paiements (commande_id, montant, mode_paiement, date_paiement)
VALUES (1,990,'Carte','2025-01-12'), (2,45,'Espèces','2025-01-21'), (3,500,'V
irement','2025-02-05');
```

---

2. Requêtes de jointures

INNER JOIN

```
SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
FROM clients c
INNER JOIN commandes cmd ON c.id = cmd.client_id
INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
INNER JOIN produits p ON dc.produit_id = p.id;
```

LEFT JOIN

```
SELECT c.nom, cmd.id AS num_commande
FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id;
```

RIGHT JOIN

```
SELECT c.nom, cmd.id, cmd.date_commande
FROM clients c
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

FULL JOIN (simulation MySQL)

```
SELECT c.nom, cmd.id AS num_commande
FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id
UNION
SELECT c.nom, cmd.id AS num_commande
FROM clients c
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

---

3. Agrégations et calculs

Montant total par commande

```
SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS total_comman
de
FROM commandes cmd
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY cmd.id;
```

Montant total dépensé par client

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
```

```sql
GROUP BY c.nom
HAVING total_client > 500;
```

Montant moyen par produit

```sql
SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen
FROM produits p
GROUP BY p.nom_produit;
```

---

4. Fonctions SQL

IF() pour classer les clients

```sql
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
       IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS
categorie
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom;
```

Fonctions de date

```sql
SELECT p.id AS paiement, cmd.id AS commande,
       DATEDIFF(p.date_paiement, cmd.date_commande) AS delai_jours
FROM paiements p
JOIN commandes cmd ON p.commande_id = cmd.id;
```

LIMIT

```sql
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom
ORDER BY total DESC
LIMIT 2;
```

---

5. IN / NOT IN / EXISTS

```sql
SELECT nom FROM clients WHERE id IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));

SELECT nom FROM clients WHERE id NOT IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));
```

```sql
SELECT nom FROM clients c
WHERE EXISTS (
  SELECT 1 FROM commandes cmd
  JOIN paiements p ON cmd.id = p.commande_id
  WHERE cmd.client_id = c.id);
```

---

6. Optimisation : Index et Vues

```sql
CREATE INDEX idx_nom_produit ON produits(nom_produit);
CREATE INDEX idx_client_ville ON clients(ville);

EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';

CREATE VIEW vue_ventes AS
SELECT c.nom, p.nom_produit, dc.quantite, (p.prix_unitaire * dc.quantite) AS
montant, pa.mode_paiement
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
LEFT JOIN paiements pa ON cmd.id = pa.commande_id;

SELECT * FROM vue_ventes WHERE montant > 200;
```

---

7. DCL – Gestion des utilisateurs

```sql
CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';

GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
FLUSH PRIVILEGES;

SHOW GRANTS FOR 'manager'@'localhost';

REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
```

Tâche finale :

Réalisez toutes les requêtes de l'atelier sur un **serveur MySQL hébergé sur Aiven** ou tout autre hébergeur MySQL gratuit.

Travail à rendre : rapport contenant

1. **Captures d'écran** prouvant vos exécutions :
    - Création des tables et insertion des données.
    - Résultat d'une jointure (INNER JOIN ou LEFT JOIN).
    - Exemple d'une requête avec GROUP BY et fonction d'agrégation (SUM, AVG ou COUNT).
    - Requête utilisant IF(), DATEDIFF() ou LIMIT.
    - Vérification de l'effet d'un **index** via EXPLAIN.
    - Lecture de la **vue** créée (SELECT * FROM vue_ventes;).
    - Application d'un droit utilisateur avec GRANT ou SHOW GRANTS.
2. **Capture du tableau de bord Aiven** prouvant la connexion réussie à la base MySQL distante.